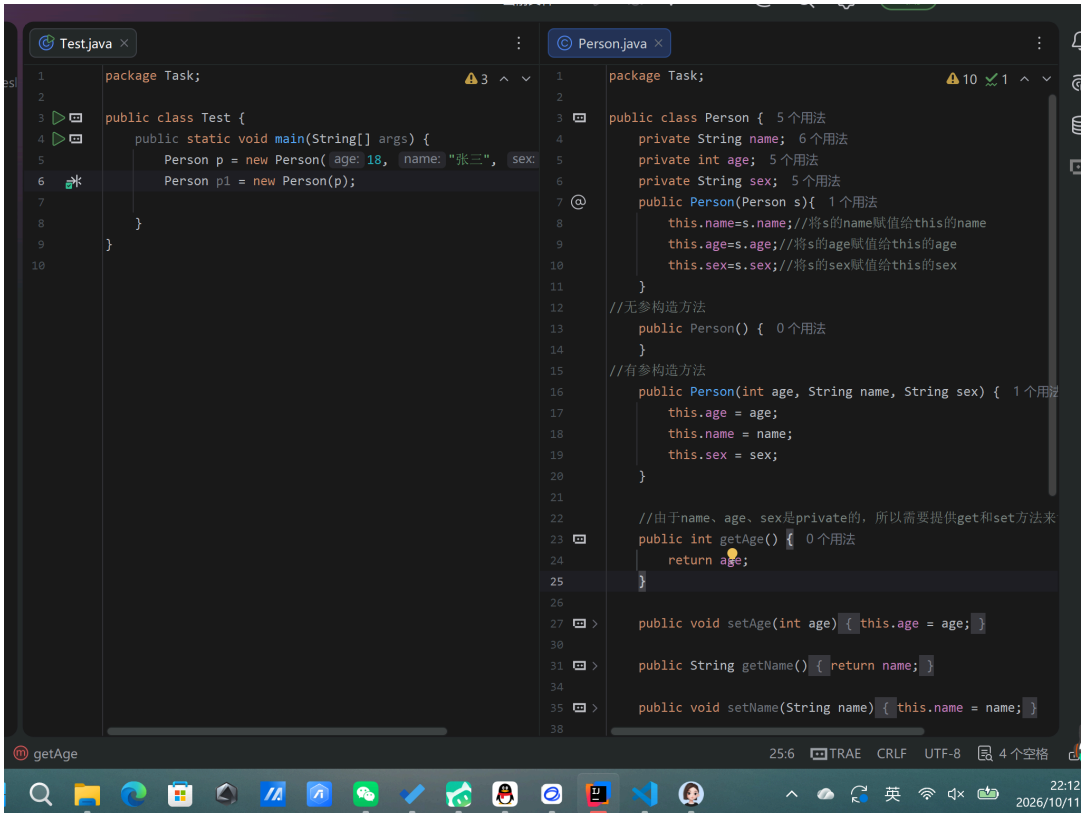
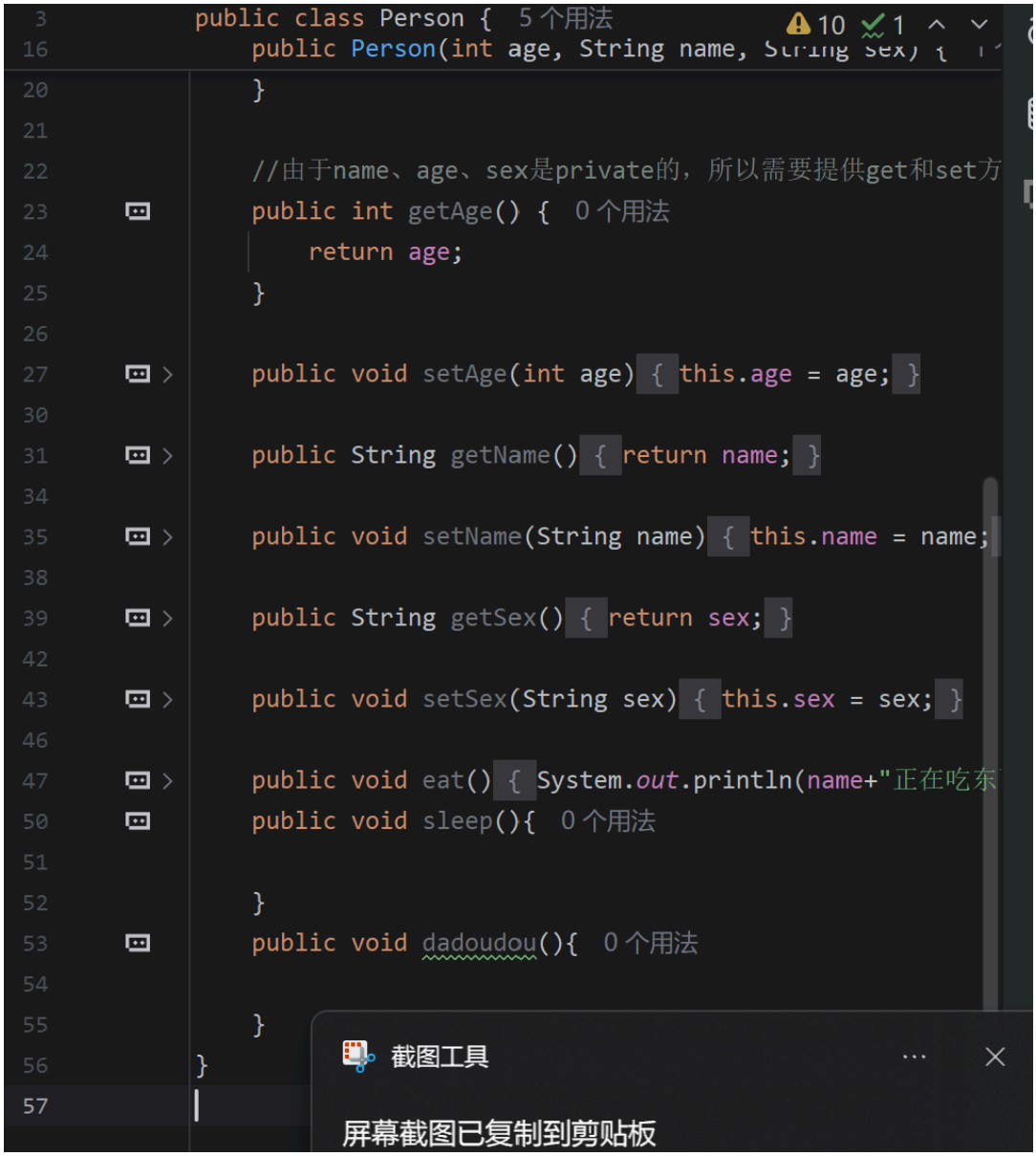
Task1

1.


Test.java
package Task;
public class Test {
public static void main(String[] args) {
Person p = new Person(age: 18, name: "张三", sex:
Person p1 = new Person(p);
}
}
Person.java
package Task;
public class Person { 5 个用法
private String name; 6 个用法
private int age; 5 个用法
private String sex; 5 个用法
public Person(Person s){ 1 个用法
this.name=s.name;//将s的name赋值给this的name
this.age=s.age;//将s的age赋值给this的age
this.sex=s.sex;//将s的sex赋值给this的sex
}
//无参构造方法
public Person() { 0 个用法
}
//有参构造方法
public Person(int age, String name, String sex) { 1 个用
this.age = age;
this.name = name;
this.sex = sex;
}
//由于name、age、sex是private的，所以需要提供get和set方法来
public int getAge() { 0 个用法
return age;
}
public void setAge(int age) { this.age = age; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
getAge
25:6
TRAE
CRLF
UTF-8
4 个空格
22:12
2026/10/11



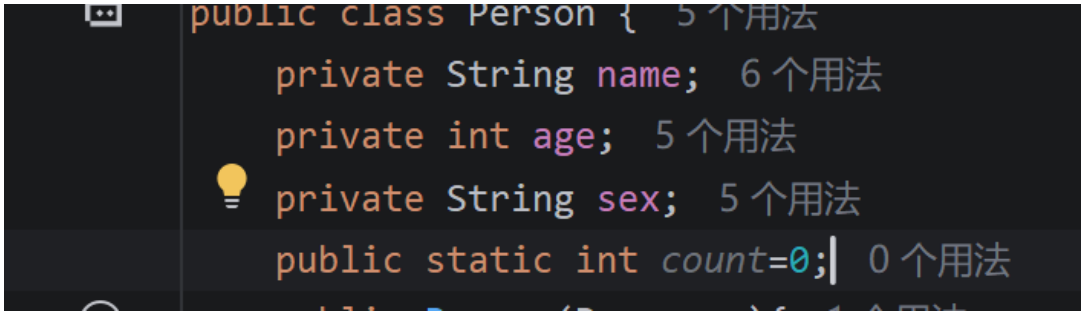
2.用了，表示当前的Person这个对象。 *3*.类里面有属性、方法，对象是以类为模板在堆中new出来的内存对应的实例；

Task2

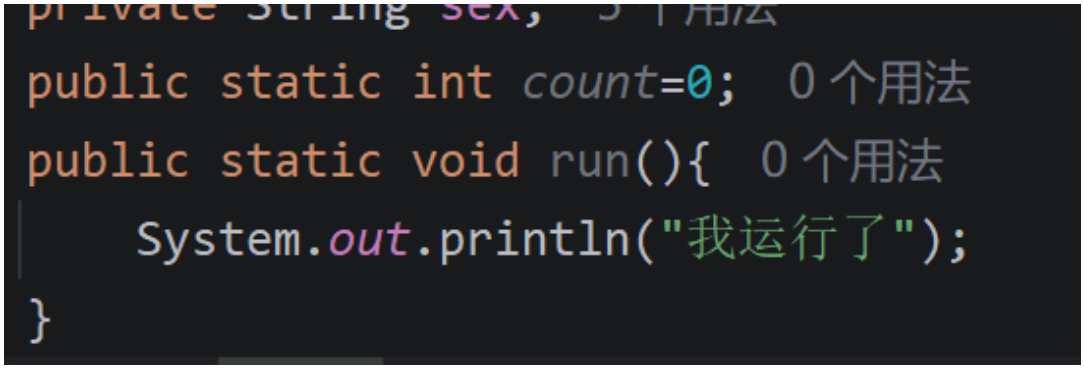
1.Task1中；

2、*3*.public在任意地方；

private在当前类； protected在本包和不同包的子类中可以访问； *4*.



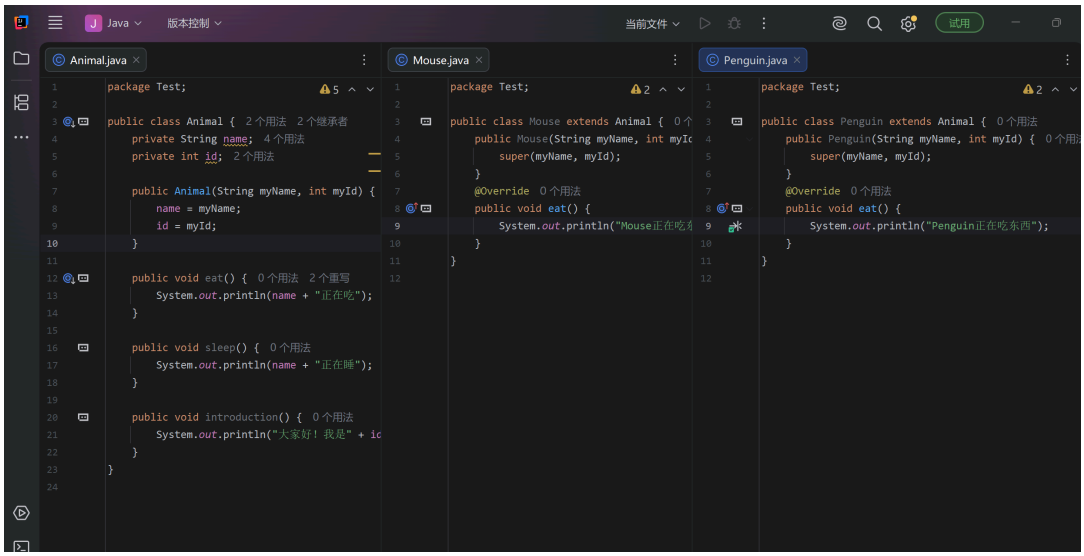
5.



1.因为静态方法在出现类名的时候就已经加载了，是跟类绑定的； 2.实例方法是在创建对象之后才有的，需要靠对象的地址去引用。

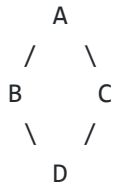
Task3

1、2、3.



*4.*如果继承的多个父类有同名的属性或方法子类不知道是哪个父类的，super

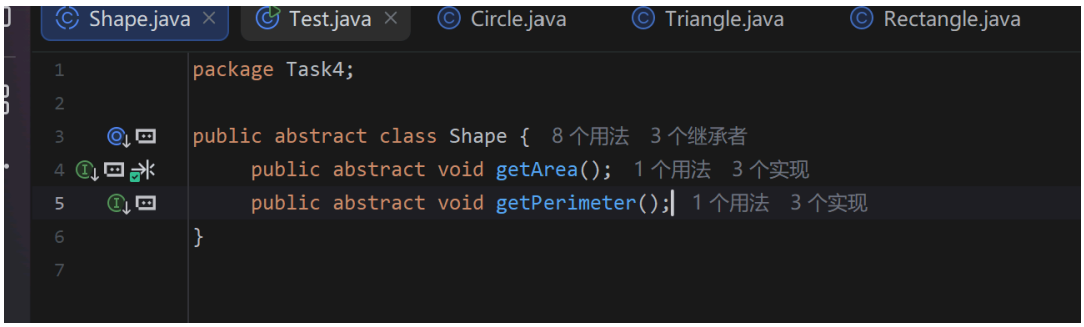
也不知道是调用的哪个父类的。
多继承问题(例如菱形继承冲突)：



这里B、C均会继承A的，未重写的部分继承，此时D会继承B、C中的属性和方法，那么就会重复继承 A中的东西了，而且如果B、C重写了A中的方法那么D就会有两个名字一样内容不一定一样的方法，此时如果引用就不知道是哪个了。

Task4

1、2



Shape.java | Test.java × | Circle.java | Triangle.java | Rectangle.java

```java
package Task4;

import java.util.ArrayList;
public class Test {
    public static void main(String[] args) {
        Shape C=new Circle( name: "圆", radius: 5);
        Shape T=new Triangle( name: "三角形", side1: 3, side2: 4, side3: 5);
        Shape R=new Rectangle( name: "矩形", length: 5, width: 4);
        ArrayList<Shape> shapes=new ArrayList<>();
        shapes.add(C);
        shapes.add(T);
        shapes.add(R);

        for(Shape test:shapes ){
            test.getArea();
            test.getPerimeter();
        }
    }
}

```

Java 版本控制

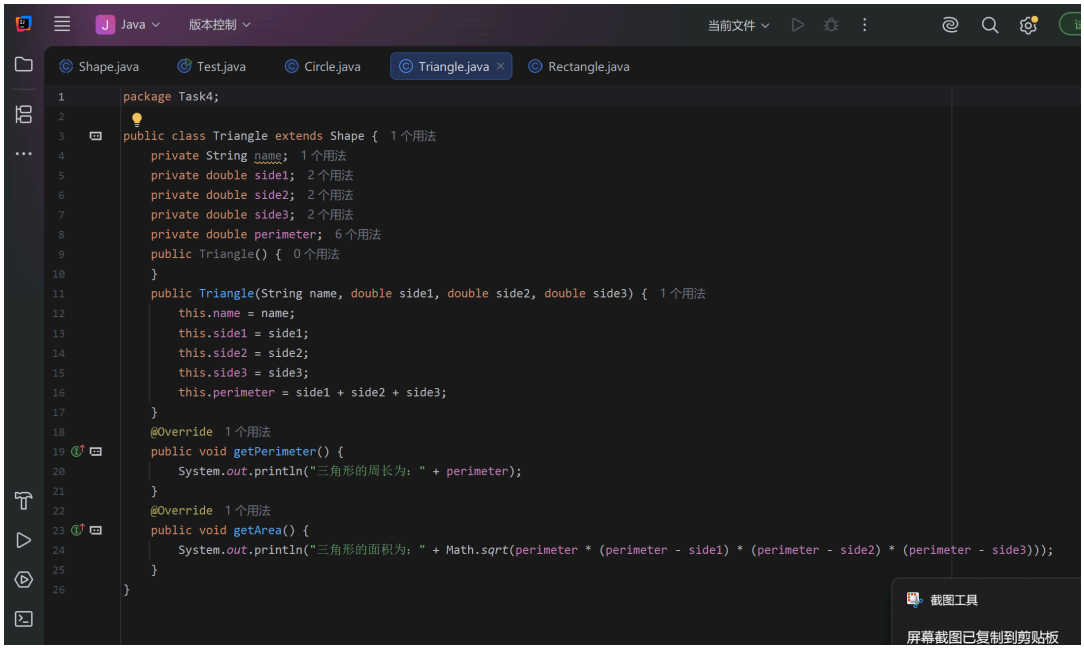
Shape.java | Test.java | Circle.java × | Triangle.java | Rectangle.java

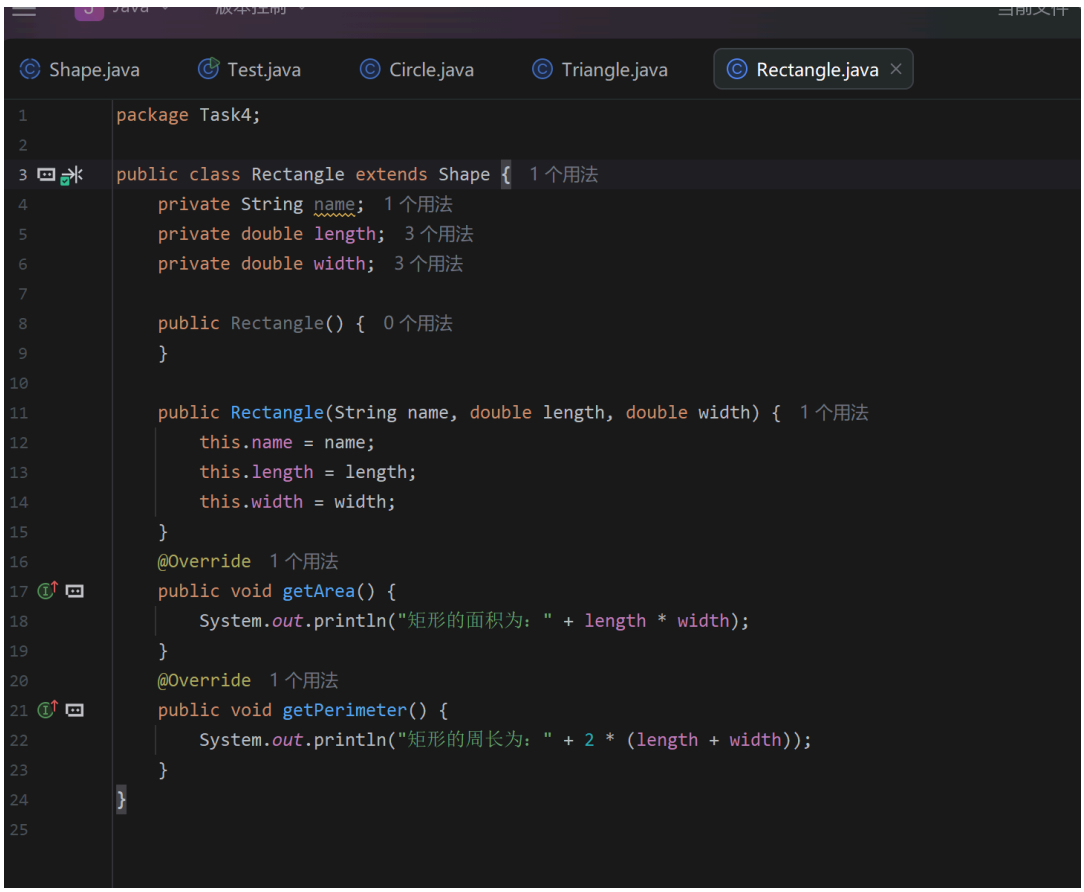
```java
package Task4;

public class Circle extends Shape {  1 个用法
    private double radius;  6 个用法
    private String name;  1 个用法
    public Circle() {  0 个用法
    }
    public Circle(String name, double radius) {  1 个用法
        this.name = name;
        this.radius = radius;
    }
    @Override  1 个用法
    public void getArea() {//Math.PI表示圆周率
        System.out.println("圆的面积为："+Math.PI*radius*radius);
    }
    @Override  1 个用法
    public void getPerimeter() {
        System.out.println("圆的周长为："+2*Math.PI*radius);//Math.PI表示圆周率;
    }

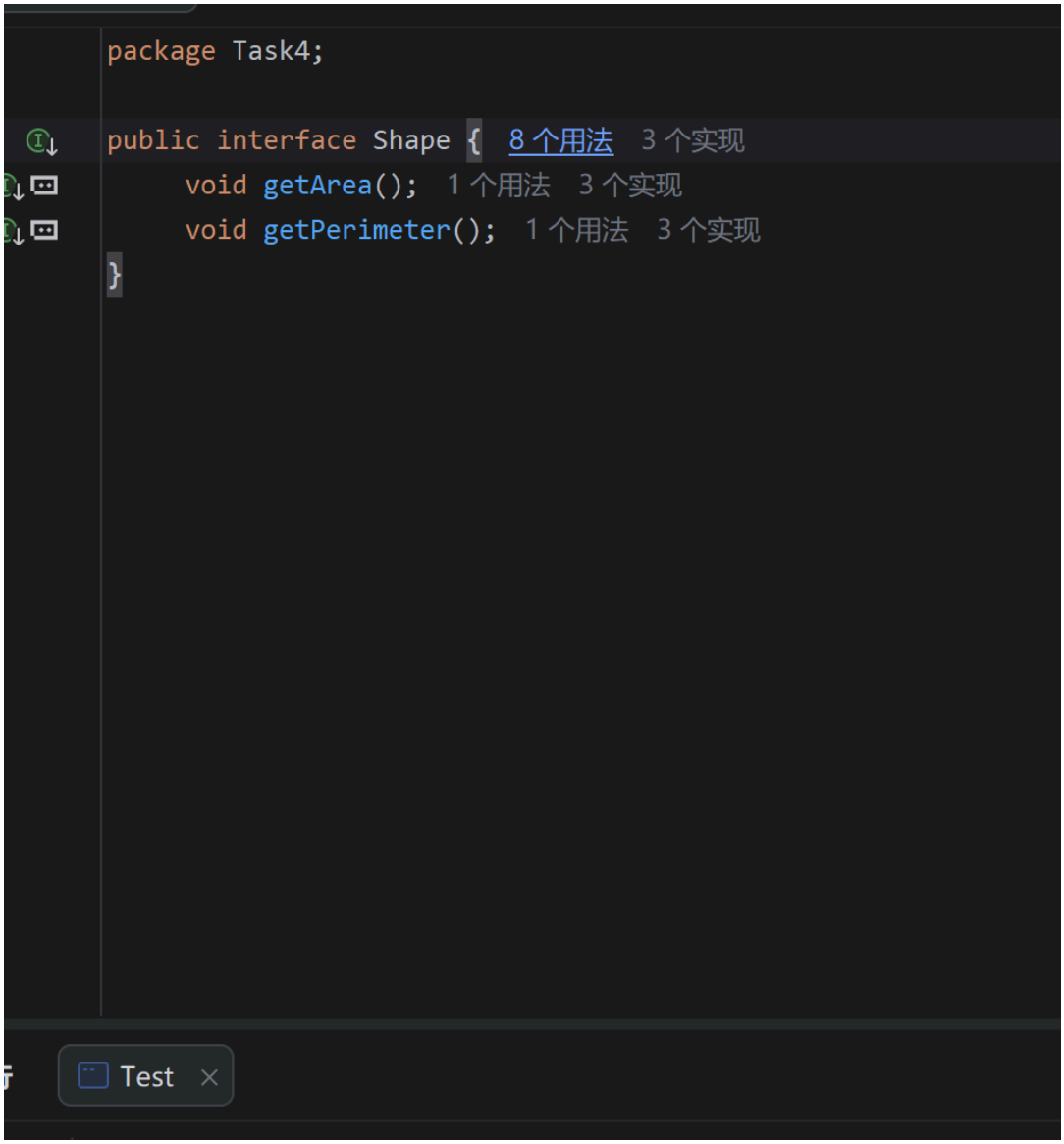
    public double getRadius() {  0 个用法
        return radius;
    }

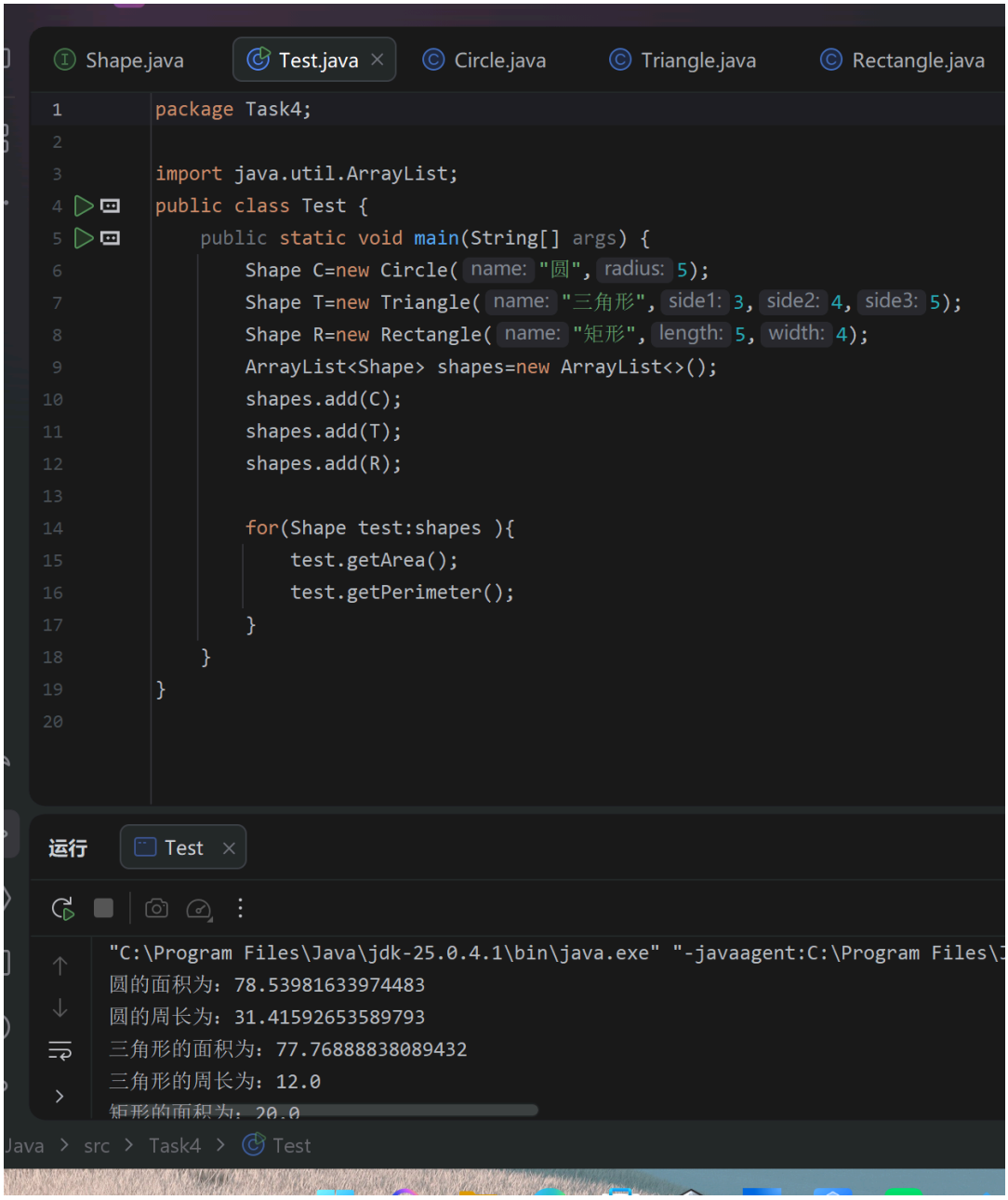
    public void setRadius(double radius) {  0 个用法
        this.radius = radius;
    }
}
```


Java
版本控制
当前文件
Shape.java
Test.java
Circle.java
Triangle.java
Rectangle.java
package Task4;
public class Triangle extends Shape { 1 个用法
private String name; 1 个用法
private double side1; 2 个用法
private double side2; 2 个用法
private double side3; 2 个用法
private double perimeter; 6 个用法
public Triangle() { 0 个用法
}
public Triangle(String name, double side1, double side2, double side3) { 1 个用法
this.name = name;
this.side1 = side1;
this.side2 = side2;
this.side3 = side3;
this.perimeter = side1 + side2 + side3;
}
@Override 1 个用法
public void getPerimeter() {
System.out.println("三角形的周长为：" + perimeter);
}
@Override 1 个用法
public void getArea() {
System.out.println("三角形的面积为：" + Math.sqrt(perimeter * (perimeter - side1) * (perimeter - side2) * (perimeter - side3)));
}
}
截图工具
屏幕截图已复制到剪贴板


Shape.java
Test.java
Circle.java
Triangle.java
Rectangle.java
package Task4;
public class Rectangle extends Shape { 1 个用法
private String name; 1 个用法
private double length; 3 个用法
private double width; 3 个用法
public Rectangle() { 0 个用法
}
public Rectangle(String name, double length, double width) { 1 个用法
this.name = name;
this.length = length;
this.width = width;
}
@Override 1 个用法
public void getArea() {
System.out.println("矩形的面积为：" + length * width);
}
@Override 1 个用法
public void getPerimeter() {
System.out.println("矩形的周长为：" + 2 * (length + width));
}
}

//


package Task4;
public interface Shape { 8 个用法 3 个实现
void getArea(); 1 个用法 3 个实现
void getPerimeter(); 1 个用法 3 个实现
}
Test


Shape.java
Test.java
Circle.java
Triangle.java
Rectangle.java
package Task4;
import java.util.ArrayList;
public class Test {
public static void main(String[] args) {
Shape C=new Circle(name: "圆", radius: 5);
Shape T=new Triangle(name: "三角形", side1: 3, side2: 4, side3: 5);
Shape R=new Rectangle(name: "矩形", length: 5, width: 4);
ArrayList<Shape> shapes=new ArrayList<>();
shapes.add(C);
shapes.add(T);
shapes.add(R);
for(Shape test:shapes){
test.getArea();
test.getPerimeter();
}
}
}
运行
Test
"C:\Program Files\Java\jdk-25.0.4.1\bin\java.exe" "-javaagent:C:\Program Files\
圆的面积为：78.53981633974483
圆的周长为：31.41592653589793
三角形的面积为：77.76888838089432
三角形的周长为：12.0
矩形的面积为：20.0
Java > src > Task4 > Test

Shape.java Test.java Circle.java Triangle.java Rectangle.java

```java
package Task4;

public class Circle implements Shape {  1 个用法
    private double radius;  6 个用法
    private String name;  1 个用法
    public Circle() {  0 个用法
    }
    public Circle(String name, double radius) {  1 个用法
        this.name = name;
        this.radius = radius;
    }
    @Override  1 个用法
    public void getArea() {//Math.PI表示圆周率
        System.out.println("圆的面积为："+Math.PI*radius*radius);
    }
    @Override  1 个用法
    public void getPerimeter() {
        System.out.println("圆的周长为："+2*Math.PI*radius);//Math.PI表示圆周率;
    }

    public double getRadius() {  0 个用法
        return radius;
    }

    public void setRadius(double radius) {  0 个用法
        this.radius = radius;
    }
}
```

Shape.java Test.java Circle.java Triangle.java Rectangle.java

```java
package Task4;

public class Triangle implements Shape {  1 个用法
    private String name;  1 个用法
    private double side1;  2 个用法
    private double side2;  2 个用法
    private double side3;  2 个用法
    private double perimeter;  6 个用法
    public Triangle() {  0 个用法
    }
    public Triangle(String name, double side1, double side2, double side3) {  1 个用法
        this.name = name;
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
        this.perimeter = side1 + side2 + side3;
    }
    @Override  1 个用法
    public void getPerimeter() {
        System.out.println("三角形的周长为：" + perimeter);
    }
    @Override  1 个用法
    public void getArea() {
        System.out.println("三角形的面积为：" + Math.sqrt(perimeter * (perimeter - side1) * (perimeter - side2) * (perimeter - side3)));
    }
}
```

截图工具

Shape.java Test.java Circle.java Triangle.java Rectangle.java

```java
package Task4;

public class Rectangle implements Shape {  1 个用法
    private String name;  1 个用法
    private double length;  3 个用法
    private double width;  3 个用法

    public Rectangle() {  0 个用法
    }

    public Rectangle(String name, double length, double width) {  1 个用法
        this.name = name;
        this.length = length;
        this.width = width;
    }
    @Override  1 个用法
    public void getArea() {
        System.out.println("矩形的面积为: " + length * width);
    }
    @Override  1 个用法
    public void getPerimeter() {
        System.out.println("矩形的周长为: " + 2 * (length + width));
    }
}

```

思路：三者都有共同的方法求面积和求周长，所以可以写在父类或者接口中，name还可以写在父类中，还有那个perimeter不能直接写在外面

```
┌──────────── 类的生命周期 ────────────┐
│ ① 加载（读 .class 文件）             │
│ ② 链接（验证/准备/解析）             │
│ ③ 类初始化 ← 静态变量在这里初始化     │ ← 执行 <clinit> 方法
│ ④ ...（之后类随时可用）              │
└──────────────────────────────────────┘

                ↓ 某个时刻有人 new

┌──────────── 对象的诞生 ──────────────┐
│ ⑤ 分配堆内存，字段填默认值（0/null）  │
│ ⑥ 对象初始化 ← 成员变量在这里初始化   │ ← 执行 <init> 方法
│     （先声明处赋值和初始化块，        │
│       再构造方法体）                 │
└──────────────────────────────────────┘
```

，之后再用多态装在集合里面再遍历

*3.*抽象类适合在几个子类有共同属性和方法的时候，此时子类应该属于同一个父类，比如学生和老师都属于人，而不是教育家
接口适合在几个类有共同的行为但是没有关系，比如人和车的都在前进。

Task5

```
public class BankAccount {
    // TODO: 修改属性的可见性
    //外界不能直接访问
    private String accountNumber;
    private String accountHolder;
    private double balance;
    private  String password;
//构造方法
    BankAccount(String accountNumber, String accountHolder, double ini
        this.accountNumber=accountNumber;
        this.accountHolder=accountHolder;
        this.balance=initialBalance;
        this.password=password;

    }
//要存钱肯定大于零所以要判断存款是否大于0
    void deposit(double amount) {
        // TODO
        if(validateAmount(amount)){
            balance+=amount;
        }else{
            System.out.println("存款金额必须大于0");
        }
    }
//要先密码对了才进行下一步，所以要判断，金额要大于零，所以要判断取款是否大
    boolean withdraw(double amount, String inputPassword) {
        // TODO
        if (validatePassword(inputPassword)) {
            if (validateAmount(balance-amount)) {
                balance-=amount;
                return true;
            } else {
                System.out.println("取款金额超出余额范围");
                return false;
```

```
            }
        }else{
            System.out.println("密码错误");
            return false;
        }
    }
//要先密码对了才进行下一步，所以要判断，金额要大于零，所以要判断转账是否大
    boolean transfer(BankAccount recipient, double amount, String inpu
        // TODO
        if (validatePassword(inputPassword)) {

            if (validateAmount(balance-amount)) {
                balance-=amount;
                recipient.deposit(amount);
                return true;
            }else{
                System.out.println("转账金额超出余额范围");
                return false;
            }
        }else{
            System.out.println("密码错误");
            return false;
        }
    }
    double getBalance() {
            return balance;
        // TODO
    }
//要先密码对了才进行下一步，所以要判断，还要隐藏信息
    String getAccountInfo() {
            String str1=accountHolder.substring(1);
            String ah1=accountHolder.replace(str1,"***");
            String str2=accountNumber.substring(3,accountNumber.length
            String ah2=accountNumber.replace(str2,"***");
            return "账号："+ah2+"\n"+"账户持有人："+ah1+"\n"+"余额："+ba
        }



    //判断密码是否正确
    private boolean validatePassword(String inputPassword) {
        if(inputPassword.equals(password)) {
            return true;
        }
        return false;
    }
```

```
    //判断金额是否大于0
    private boolean validateAmount(double amount) {
        if(amount>0){
            return true;
        }else{
            return false;
        }
    }
```

不足：因为不确定名字和账号的长度，所以就不能完全做好隐藏的大小